TEAM PHOTOSPHERE · RESEARCH REPORT

SolarDoctor Landscape Study

Existing GitHub projects, startups, prototypes and UI/UX references for AI-driven solar PV fault diagnosis and O&M — and where SolarDoctor can win.

SolarDoctor faces fragmented rivals, commoditized vision

0
projects found that combine all four SolarDoctor pillars

24
GitHub repos cloned & inspected

92.37%
best published accuracy on the same 6 classes

8+
near-identical 6-class student classifiers

Prepared for Parth Gawade
B.Tech CSE (AI & ML) · PCCoE, Pune

3 November 2026
13 sections · verified sources

Contents

01 Executive Summary

02 Best 10 GitHub Projects

03 Best Full-Stack Projects

04 Best UI/UX References

05 Feature-by-Feature GitHub Mapping

06 Other Web Projects / Startups / Hackathon Projects

07 Competitive Landscape

08 What Existing Projects Do Better Than My Idea

09 What My Idea Can Do Better

10 Top 5 Code References

11 Top 5 UI/UX References

12 Top 3 Repositories to Use as a Foundation

13 Recommended Development Strategy Based on This Research

Conclusion

Sources

RESEARCH METHOD & VERIFICATION LIMITS

Research method & verification limits. GitHub stars, forks, last-push dates and licenses come from the GitHub Search API (`gh search repos` / `search/repositories`) queried on **2026-11-03**. The `repos/OWNER/REPO` endpoint returned HTTP 403, so the search API was used instead. Around 24 repos were shallow-cloned and grepped for `lime` , `grad-cam` , `pvlib` , `clearsky` , `nasa power` , `pvdaq` , `work order` , `vgg16` and `autoencoder` . Library function signatures (pvlib 0.16.1, pvanalytics 0.2.2, rdtools 3.2.1, pvdaq_access 0.1.2) were checked against the installed PyPI packages. **No live demos were loaded.** vercel.app, streamlit.app, github.io, Hugging Face listings, Kaggle, Devpost, Product Hunt and most vendor homepages were blocked by the proxy or allowlist, so every demo URL below is marked **unverified**. **No screenshots were taken.** UI/UX ratings come from vendor help-center documentation, marketing pages and README references, not from viewing the products. Accuracy figures are self-reported by each repo or paper and were not reproduced, and no code was executed end to end. Throughout, **Verified** means read in a source this session. **Assessment** means the analyst's judgment.

Green licence — reusable | Amber — AGPL / unknown, check | Red — no licence, study only

01 Executive Summary

No open-source project, startup or academic prototype combines all four SolarDoctor pillars, which are a physics-baseline yield deficit, a 6-class RGB defect classifier, LIME explanations, and ₹ ROI with work-order dispatch. The pieces all exist, scattered across 0–2-star student and portfolio repos and enterprise products. On the image side, **mcomuniello/Apollo9** is the closest match. It uses the same six Kaggle classes, adds LIME, Grad-CAM and SHAP, and runs on FastAPI, Streamlit and Docker, but it has no license (Apollo9). On the telemetry side, **anugrahpatil-creator/SolarPulseAI** compares a pvlib physics baseline against SCADA data for Indian solar parks. It is MIT-licensed (SolarPulseAI). On the O&M side, **VishnuBhaiya/solarvision-inspection** ships a work-order lifecycle with ROI estimates, which its own README calls illustrative (solarvision-inspection).

The 6-class RGB classifier is the weakest differentiator. Dozens of near-identical repos use the same Kaggle taxonomy, mostly scoring 83–90% (GitHub search). A 2026 paper reports **92.37%** for HybridSolarNet and **90.84%** for plain EfficientNet-B0, on a dataset of only **~875 raw images**. So SolarDoctor's 91.46% is competitive, not superior (arXiv 2601.02928).

The real opening is in the coupling. No inspected repo uses **NASA POWER or PVDAQ inside an app**. None pairs a telemetry deficit with an image diagnosis, and none turns the result into a rupee cleaning decision for small rooftop and C&I owners, a market that enterprise vendors such as Raptor Maps and Zeitview do not serve.

The recommended build has four parts. Use **pvlib + rdtools + pvanalytics** (BSD/MIT) as the physics core. Re-train the classifier on **224×224 RGB** with a leak-free split. Add Grad-CAM alongside LIME. Copy the Sitemark/Raptor anomaly-panel layout and the Zeitview severity scales for the UI.

02 Best 10 GitHub Projects

Selection criterion: overlap with SolarDoctor's combined feature set, weighted toward working full-stack apps. All stats are **Verified** (GitHub Search API, 2026-11-03). Rows marked *(assessment)* are analyst judgment.

2.1 mcomuniello/Apollo9

closest image-side twin

GITHUB	github.com/mcomuniello/Apollo9
LIVE DEMO	None found
TECH STACK	Python; FastAPI `main.py` (`/health` , `/predict/rgb` , `/predict/thermal`); Streamlit `app.py` / `app_api.py` ; `Dockerfile.api` , `Dockerfile.streamlit` , `docker-compose.yml`
WHAT IT DOES	"AI-powered RGB and thermal solar panel inspection platform with defect classification, thermal anomaly screening, explainable AI, reporting" (repo)
OVERLAP	Uses **exactly SolarDoctor's six classes**: Bird-drop, Clean, Dusty, Electrical-damage, Physical-Damage, Snow-Covered (inference.py). LIME through `lime.lime_image` with SLIC superpixels, optional SHAP, a Grad-CAM path, `build_pdf_report` , triage badges, health gauges, and an analysis history (explainability.py, app.py)
DIFFERENCES	Has no telemetry or clear-sky baseline, no kWh/₹ ROI and no work orders. Adds a thermal tab
UI/UX QUALITY	Not observed. The README is one line with no screenshots
ARCHITECTURE WORTH STUDYING	The API/UI split, with separate Dockerfiles for the model server and the Streamlit client
STARS / FORKS / ACTIVITY	★0 / 1 fork / created and last pushed 2026-05-06 (a single-commit snapshot)
MAINTAINED?	No activity since its first push *(assessment: dormant snapshot)*
TAKE INSPIRATION FROM	`explainability.py` (LIME with SLIC parameters), the side-by-side multi-XAI tabs, and the PDF report builder
LICENSE	**None. Not legally reusable**; study only

2.2 anugrahpatil-creator/SolarPulseAI

closest telemetry-side twin, Indian context

GITHUB	github.com/anugrahpatil-creator/SolarPulseAI
LIVE DEMO	solar-pulse-ai-one.vercel.app (**unverified**)
TECH STACK	FastAPI (`backend/app` , `backend/ml` , pytest), Next.js 15, PostgreSQL + TimescaleDB, Docker Compose + nginx, `render.yaml` . pvlib appears in 3 files and clear-sky references in 15

2.2 anugrahpatil-creator/SolarPulseAI

closest telemetry-side twin, Indian context

WHAT IT DOES	Combines "solar physics (pvlib) with gradient boosting (LightGBM)" for AC forecasts. A "live anomaly engine" compares the forecasts with SCADA telemetry and "surfaces operational shortfalls" (README)
OVERLAP	Physics baseline vs actual output, which is SolarDoctor's pillar (a). Also has dynamic soiling estimation (time since rain, cleaning logs), inverter clipping, a configurable alert threshold (`/api/v1/anomaly/{plant_id}/alerts`, `/threshold`), and an alert console with "suggested remedies". It comes pre-seeded with **Bhadla (50 MW), Pavagada (25 MW) and Kamuthi**
DIFFERENCES	No computer vision, no LIME, no work orders. ROI appears only as "revenue loss"
UI/UX QUALITY	Not observed; no screenshots were inspected
ARCHITECTURE WORTH STUDYING	A TimescaleDB time-series store and a threshold-configurable anomaly API
STARS / FORKS / ACTIVITY	★1 / 2 forks / last push 2026-10-17
MAINTAINED?	Active
TAKE INSPIRATION FROM	The soiling-since-rain feature, the threshold endpoint design, and seeding with Indian plants for the demo narrative
LICENSE	**MIT** (reusable with attribution)

Caveat *(assessment)*: the repo carries `.claude/` / `AGENTS.md`-style files and polished boilerplate. It is likely an AI-assisted 2026 portfolio project, so treat its README claims with caution.

2.3 VishnuBhaiya/solarvision-inspection

work-order lifecycle reference

GITHUB	github.com/VishnuBhaiya/solarvision-inspection
LIVE DEMO	None found
TECH STACK	Flask, SQLite, PyTorch detectors (YOLO, SAHI, Faster R-CNN, SSD), JS (`charts.js`, `pdf-export.js`), CI in `.github/workflows/checks.yml`, unit tests, and a model manifest that SHA-256-verifies downloaded weights
WHAT IT DOES	"AI-assisted solar-panel inspection with a Flask dashboard, two-stage panel localisation and defect detection, image/video workflows, maintenance work orders and inspection reports." It has 8 defect categories and optional DJI Tello drone integration (README)
OVERLAP	Defect detection (b), plus work orders, scheduling and ROI-like estimates (d). Routes include `/workorders`, `/api/workorders/<wo_id>/complete`, `/schedule`, `/analytics`, `/report`, and `/api/export/csv|json` (app.py)
DIFFERENCES	No telemetry baseline and no LIME. The README states: "Simulated defects and energy/ROI estimates are illustrative, not measured engineering results"

2.3 VishnuBhaiya/solarvision-inspection

work-order lifecycle reference

UI/UX QUALITY	Not observed. Templates exist (`workorders.html`, `schedule.html`, `analytics.html`)
ARCHITECTURE WORTH STUDYING	A simple Flask + SQLite work-order state machine, and two-stage localisation followed by classification
STARS / FORKS / ACTIVITY	★0 / 0 / last push 2026-10-29
MAINTAINED?	Active
TAKE INSPIRATION FROM	The work-order completion endpoint, the weight-integrity manifest, and CSV/JSON/PDF export
LICENSE	**None. Not legally reusable**

2.4 rithvik123/solar-panel-defect-detection

best-engineered image pipeline

GITHUB	github.com/rithvik123/solar-panel-defect-detection
LIVE DEMO	None found
TECH STACK	Python (~420 files). FastAPI v1 (`api/routes`, middleware, schemas, lifespan), Streamlit dashboard (`dashboard/app.py`, `api_client.py`), Docker Compose + Nginx, per-stage YAML configs, CHANGELOG, CITATION.cff, SECURITY.md
WHAT IT DOES	"Solar Panel Defect Detection and Maintenance Decision Support". It runs an EfficientNet-B0 global classifier, a **clean-panel anomaly model with a heatmap**, candidate-region generation, weak-region multiple-instance learning (MIL), and pseudo-mask refinement. Each case gets "risk, confidence, priority and maintenance guidance" and is assigned a maintenance priority band (README)
OVERLAP	Same Kaggle-style RGB taxonomy. Its anomaly model is conceptually close to SolarDoctor's autoencoder. Also does decision support
DIFFERENCES	No telemetry, no ₹ ROI, no LIME. Trained on weak folder labels only
UI/UX QUALITY	The README references a dashboard screenshot (`docs/assets/screenshots/streamlit/hero-dashboard.png`); it was not viewed
ARCHITECTURE WORTH STUDYING	**The FastAPI + Streamlit split that SolarDoctor's stack most resembles**, and config-driven stages
STARS / FORKS / ACTIVITY	★1 / 0 / last push 2026-07-19
MAINTAINED?	Moderately active *(assessment)*
TAKE INSPIRATION FROM	The `api_client.py` pattern (Streamlit calls FastAPI, never the model directly), YAML stage configs, and the priority-band output

2.4 rithvik123/solar-panel-defect-detection

best-engineered image pipeline

LICENSE	"Selection pending", i.e. **none. Not legally reusable**

2.5 Parakh20/AxalonSystems

most production-grade architecture

GITHUB	github.com/Parakh20/AxalonSystems
LIVE DEMO	axalon-systems.vercel.app (**unverified**)
TECH STACK	YOLO11x, FastAPI, SQLAlchemy + Alembic (SQLite locally, Postgres in production), Next.js 14 dashboard at `/platform`, CI, and deploy configs for Azure, GCP, Fly and HF
WHAT IT DOES	"Drone-based thermal inspection platform for solar parks" with 11 thermal fault classes (including soiling), thermal/RGB fusion, and panel localisation by OCR plus an auto-grid. Reports export as JSON, Excel, GeoJSON and PDF (README)
OVERLAP	Defect detection (b, thermal rather than RGB) and work orders (d) through `platform/api/routers/work_orders.py` and `platform/core/fault_workflow.py`
DIFFERENCES	Thermal-first. No telemetry baseline, no LIME, no ₹ ROI
UI/UX QUALITY	Not observed
ARCHITECTURE WORTH STUDYING	A layered `ml / platform / website / drone` monorepo, Alembic migrations, and a fault-to-work-order workflow module
STARS / FORKS / ACTIVITY	★0 / 0 / last push 2026-10-14
MAINTAINED?	Active
TAKE INSPIRATION FROM	The `fault_workflow.py` state transitions, GeoJSON export for map layers, and panel localisation by grid
LICENSE	**MIT**

2.6 VisionExpo/solar-panel-fault-detection

TensorFlow + LIME/SHAP API twin

GITHUB	github.com/VisionExpo/solar-panel-fault-detection
LIVE DEMO	None found in the README
TECH STACK	**TensorFlow** EfficientNetB3 (384×384), FastAPI REST, Gradio UI, Docker, Render deployment, a 45 MB quantised model
WHAT IT DOES	6-class fault classification. Claims ~87% accuracy and F1 ~0.85, trained on a Kaggle "Solar Augmented Dataset" (repo)

2.6 VisionExpo/solar-panel-fault-detection

TensorFlow + LIME/SHAP API twin

OVERLAP	6 classes and **a LIME and SHAP explanation endpoint** (`{"method": "lime"}`), in a web app
DIFFERENCES	No telemetry, no ROI, no work orders
UI/UX QUALITY	Not observed
ARCHITECTURE WORTH STUDYING	An explanation method chosen as an API parameter, plus a quantised model for cheap hosting
STARS / FORKS / ACTIVITY	★4 / 0 / last push 2026-11-02
MAINTAINED?	Active
TAKE INSPIRATION FROM	**It is the closest stack match to a TF/Keras team.** Look at the explanation endpoint contract, quantisation and the Render deploy
LICENSE	**Apache-2.0** (reusable; keep the NOTICE file and mark your changes)

2.7 toannnnq1424/solar-fleet-ems

real inverter ingestion and work-order guards

GITHUB	github.com/toannnnq1424/solar-fleet-ems
LIVE DEMO	None found
TECH STACK	Python FastAPI, JS ES modules, SQLite WAL (~337 files). README in Vietnamese and English
WHAT IT DOES	A "local-first multi-vendor solar O&M and EMS platform" that reads 8 vendor clouds (Deye, SolisCloud, SOLARMAN, GoodWe, Sungrow, Huawei, Growatt, Eybond). The README calls it "pilot 0.2.0, not yet a mature product" (README)
OVERLAP	A partial baseline: `weather.py` has clear-sky references, and `forecast_baseline.py` is an "explainable hourly baseline" adapted from MIT-licensed sem-community code. Also work orders and incidents: `maintenance.py` has JobPlan/JobStep and `guard_work_order_completion`, plus `incident_api.py` (maintenance.py, forecast_baseline.py)
DIFFERENCES	No image CV, no LIME
UI/UX QUALITY	Not observed
ARCHITECTURE WORTH STUDYING	Vendor adapter modules, and **guards that block work-order completion until conditions are met**
STARS / FORKS / ACTIVITY	★0 / 0 / last push 2026-10-09
MAINTAINED?	Active pilot

2.7 toannnnq1424/solar-fleet-ems

real inverter ingestion and work-order guards

TAKE INSPIRATION FROM	Inverter-API adapter abstraction (the path from a PVDAQ demo to real Indian rooftop inverters such as Growatt and Solis) and the job-step model
LICENSE	**None. Not legally reusable**

2.8 jkumarr10/veris-hackathon

cleaning-ROI decision agent

GITHUB	github.com/jkumarr10/veris-hackathon
LIVE DEMO	None found
TECH STACK	FastAPI, OpenAI Agents SDK, React (Vite)
WHAT IT DOES	"Solar Soiling Detection Agent". It geocodes the farm address, pulls Open-Meteo weather, air-quality and rainfall data, and analyses yield from a CSV. It returns `deploy_crew`, `wait_and_monitor` or `alert_only`, along with "projected loss if not cleaned", "projected gain if cleaned (fixed 30-day horizon)", "cleaning cost" and "ROI ratio" (README)
OVERLAP	**The closest match to SolarDoctor's recoverable-kWh to dispatch logic**
DIFFERENCES	Uses Open-Meteo rather than NASA POWER, and has no CV or LIME
UI/UX QUALITY	Not observed
ARCHITECTURE WORTH STUDYING	A three-way decision enum with explicit economics attached to each decision
STARS / FORKS / ACTIVITY	★0 / last push 2026-04-18
MAINTAINED?	Hackathon artifact, likely finished *(assessment)*
TAKE INSPIRATION FROM	The decision output schema. Copy the field names, not the code
LICENSE	**None. Not legally reusable**

2.9 Princeg1204/solar-panel-defect-classifier ("SolarScan AI")

honest accuracy ladder and OOD guard

GITHUB	github.com/Princeg1204/solar-panel-defect-classifier
LIVE DEMO	solar-panel-defect-classifier-vumzeomsekacrvacjkq5tu.streamlit.app (**unverified**)
TECH STACK	Baseline CNN, MobileNetV2, EfficientNetB0, Optuna, ensemble + TTA, Grad-CAM, Streamlit

2.9 Princeg1204/solar-panel-defect-classifier ("SolarScan AI")

honest accuracy ladder and OOD guard

WHAT IT DOES	6-class classification with a documented model progression: **60.9% → 74.1% → 79.9% → 83.3% → 85.6%** validation accuracy. **Out-of-distribution detection rejects non-panel images** (README)
OVERLAP	Classifier plus XAI
DIFFERENCES	Uses Grad-CAM, not LIME. Classifier only
UI/UX QUALITY	Not observed
ARCHITECTURE WORTH STUDYING	The OOD rejection step before classification
STARS / FORKS / ACTIVITY	★1 / last push 2026-08-23
MAINTAINED?	Recently active
TAKE INSPIRATION FROM	**Publish an ablation ladder like this one**, and add an OOD gate so a selfie is not classified as "Dusty"
LICENSE	**None. Not legally reusable**

2.10 dunglt69/solar-anomaly-detection ("EnergiaMind")

thesis-grade telemetry HMI and ticketing

GITHUB	github.com/dunglt69/solar-anomaly-detection
LIVE DEMO	None found
TECH STACK	TypeScript, Vite + React TS client, InceptionTime 1D-CNN through onnxruntime-node
WHAT IT DOES	A HUST thesis project, described in the README as "a fictional platform ... for academic research". It has an ISA-101 HMI dashboard and classifies DC V/I, irradiance and temperature (13 features) into 5 classes: Normal, Short-Circuit, Degradation, Open Circuit, Shadowing. "Automated Incident Ticketing" creates an alert plus a ticket and broadcasts it live (README)
OVERLAP	Telemetry fault detection (a) and ticketing (d)
DIFFERENCES	No images, no LIME, no ROI
UI/UX QUALITY	Not observed. The ISA-101 HMI convention is a deliberate industrial-UI choice
ARCHITECTURE WORTH STUDYING	ARCHITECTURE.md, AI_ARCHITECTURE.md, UML use-case and sequence diagrams, and server-side downsampling for charts
STARS / FORKS / ACTIVITY	★1 / 1 fork / last push 2026-11-02
MAINTAINED?	Active

2.10 dunglt69/solar-anomaly-detection ("EnergiaMind")

thesis-grade telemetry HMI and ticketing

TAKE INSPIRATION FROM	The documentation style for a judging panel, and **electrical fault classes inferred from V/I data rather than RGB**
LICENSE	**Apache-2.0**

Also worth knowing. P-Darabi/SolarPanelsDefectDetector (★21, MIT) is the highest-starred fault-detection app found. It runs YOLOv9 thermal detection behind FastAPI, with mAP@0.5 of 78% (README). **4keles/Solar-Panel-AI-Analysis** (★2) is a YOLO11 RGB+thermal PyQt6 desktop app with Label Studio active learning and versioned HF models (README). **EfeCicekdagi/Solar-Panel-Fault-Detection-with-YOLOv11-nano** (★12, MIT) is the highest-starred repo built on the same dataset, but it is training scripts only (repo).

03 Best Full-Stack Projects

Project	Frontend	Backend	DB	Auth	AI/ML	External APIs	Deployment	Useful parts for SolarDoctor
AxalonSystems (MIT)	Next.js 14 operator dashboard (`/platform`)	FastAPI, routers incl. `work_orders.py`	SQLAlchemy + Alembic; SQLite locally, Postgres in prod	Not documented in notes	YOLO11x thermal, 11 classes; RGB/thermal fusion; OCR grid localisation	Drone module	Azure, GCP, Fly and HF configs; CI	Layered monorepo, migrations, fault-to-work-order workflow, GeoJSON export
SolarPulse AI (MIT)	Next.js 15	FastAPI (`backend/app`, `backend/ml`)	PostgreSQL + TimescaleDB	Not documented	pvlib physics + LightGBM; anomaly engine	SCADA telemetry	Docker Compose + nginx; `render.yaml`	Physics-vs-actual anomaly API with configurable thresholds; time-series DB choice
rithvik123 (no license)	Streamlit (`dashboard/app.py`)	FastAPI v1 with middleware, schemas and lifespan	Not documented	Not documented	EfficientNet-B0 + clean-panel anomaly model + MIL	—	Docker Compose + Nginx	The Streamlit-to-FastAPI client pattern your stack needs (study only)
solarvision-inspection (no license)	Flask Jinja templates, `charts.js`, `pdf-export.js`	Flask	SQLite	Not documented	YOLO / SAHI / Faster R-CNN / SSD	DJI Tello (optional)	CI workflow	Work-order lifecycle and PDF/CSV export (study only)
Apollo9 (no license)	Streamlit	FastAPI	None (session history)	None	6-class CNN + LIME / SHAP / Grad-CAM	—	Two Dockerfiles + compose	XAI module layout (study only)
solar-fleet-ems (no license)	JS ES modules	FastAPI	SQLite WAL	Not documented	Explainable hourly baseline	8 inverter vendor clouds	Local-first	Vendor adapters and work-order guards (study only)

Project	Frontend	Backend	DB	Auth	AI/ML	External APIs	Deployment	Useful parts for SolarDoctor
EnergiaMind (Apache-2.0)	Vite + React TS (ISA-101 HMI)	Node / TypeScript	Not documented	Not documented	InceptionTime 1D-CNN through ONNX	—	Not documented	ONNX-in-Node inference and live incident broadcast
xenofex7/solar-tracker (MIT)	Flask templates	Flask JSON API + bundled MCP server	SQLite	**Admin / read-only roles**	None (target vs actual)	Home Assistant LTS, Fronius Solar.web	Docker; homepage xenofex7.github.io/solar-tracker (unverified)	The only project with a documented role model; actual-vs-target yield and amortisation

Assessment. Copy SolarPulseAI's pattern for the telemetry half and rithvik123's pattern for the image half, joined by one FastAPI service. Auth is undocumented almost everywhere. For a hackathon, a single demo login is enough. xenofex7's admin/read-only split is the simplest model worth copying later.

04 Best UI/UX References

Evidence status. No product UI was directly observed. No screenshots were captured, and the `screens/` folder is empty. Every finding below comes from vendor user documentation [DOC], which names real UI controls but was not seen on screen; from marketing pages [MKT]; or from repo metadata [META]. Typography, colour hex values, density, mobile layouts, micro-interactions and empty/loading/error states are **unknown** for all commercial products.

S Tier

Reference	Link	Evidence	What to take
Raptor Maps Digital Twin	Knowledge Hub	DOC + published GIFs (Bulk update map selection GIF)	Site Dashboard / Map View / List View tri-view. **Numbered anomaly-count bubbles per block that update live with filters**. Shift-drag box-select feeding a "Bulk Action" button. An Anomaly Change Log
Sitemark anomaly analysis	Support article	DOC	Right-side **anomaly detail panel**: classification → severity → evidence (ΔT) → **estimated energy yield loss** → location → history (new vs recurring) → remedial plan. A data-layers panel. "Swipe Compare" curtain. "Charts Mode"
Sitemark remedial actions	Support article	DOC	Action vocabulary: **Replace, Repair, Quick Fix, Move, Clean, Remove Vegetation, Field Check, Monitor, Warranty, Do Nothing**. Built-in technician times. An "Estimated Remedial Cost" field. Status tracking. Field Link sync back from technicians
Zeitview Comparison View	User guide	DOC	Compare-status legend: **New (red), Fixed (green), Persistent (yellow), Improved (light green), Degraded (orange), Reclassified (purple)**. Power loss shown in % and kW with trend arrows. The older inspection always sits on the left
Zeitview severity & condition	User guide	DOC	**3-tier severity** (1 High, 2 Medium, 3 Lowest). **5-grade site condition from % power loss** for sites ≤5 MW DC: Very Good 0–0.3%, Good 0.3–1%, Fair 1–2%, Poor 2–6%, Very Poor 6%+
Home Assistant Energy cards	Docs	DOC	`energy-solar-graph` **overlays forecast on actual**, the same visual problem as actual vs clear-sky. One global date picker with "compare to previous period". Sankey flow. Self-sufficiency gauges

A Tier

Reference	Link	Evidence	What to take
Tremor	tremor.so, repo, RedwoodJS how-to	DOC/MKT	KPI pattern: `Card` > `Metric` + `BadgeDelta` > `ProgressBar` against a target. 35+ React/Tailwind components, **including loading, empty and error states**
Roboflow confidence slider	Docs	DOC	A live "Confidence Threshold" slider that re-renders predictions as it moves
Raptor Maps what's new	Blog	MKT/blog	Mobile field map, as-built transparency, a mini-map, and Report Comparison (new vs chronic)
Sitemark saved filters	Support article	DOC	Team-shared saved filters (example: "Hotspots with ΔT > 10 °C") and share-as-link
Power Factors Unity	Solar Power World	MKT	"Detailed energy loss allocation" and fleet-level issue prioritisation

B Tier

Reference	Link	Evidence	What to take
Sitemark operations	Marketing page	MKT	Hero images of an "operations dashboard" and a "work management board"
Enphase public view	Support	DOC	A shareable read-only site page showing the array layout
HACS energy-flow cards	lumina-energy-card (315★), energy-flow-card-plus	META	Animated energy-flow visuals
Solar Forecast Arbiter	Repo	META	Forecast-vs-actual metric reports
Grafana PV dashboards	markusdd/solar_dashboard (70★), Grafana 17503	META	Dense time-series panels
Interactivity x Explainability study	Summary	Summary only	Tested static vs filtering, overlay and counterfactual explanations (N=24). Results were not available

Pattern analysis (all *assessment*, derived from the documentation above)

Landing. The commercial leaders lead with portfolio KPIs, not a hero image. The SolarDoctor landing screen should therefore be a portfolio overview: a row of Tremor-style KPI cards (energy today vs expected, deficit %, ₹ lost this month, open work orders by severity) above a site table. Each site row gets a Zeitview-style condition-grade chip and a sparkline.

Navigation and IA. All three leaders converge on **portfolio → site → inspection → anomaly** (Raptor, Zeitview). Suggested layout: a left rail (Overview · Sites · Diagnose · Work Orders · Reports), a top bar with the site

selector and an HA-style date picker, and a ~420 px right drawer for details. On mobile, use a bottom tab bar and bottom sheets.

User flows. The core SolarDoctor flow is deficit alert → "inspect" → photo upload → diagnosis + LIME → ROI card → "Create work order" → technician closes it with an after-photo → the deficit chart confirms recovery. No competitor documents that full loop as one flow.

Components. The documented components to reuse are an anomaly drawer in Sitemark's section order, a count-bubble map marker, a bulk-action bar, a change-log timeline, a curtain compare slider and a confidence bar list.

Data visualisation. Draw actual output as a line over a clear-sky expected **band**, with the deficit area shaded in the severity colour (following HA's forecast overlay). Add a loss-attribution bar list covering soiling, shading, defects, inverter and unknown (following Power Factors).

Colour. Severity is red, amber, yellow, blue-grey, then green for resolved. Reuse Zeitview's compare legend as is. Keep the LIME/Grad-CAM colormap (viridis-like) **separate from the severity palette**, so a red heatmap region is never read as "critical". Pair every colour with an icon and a label.

AI interaction patterns. No solar vendor documents a LIME/Grad-CAM UI. Showing explanations is therefore a differentiator for SolarDoctor, not an industry convention. Build in a heatmap opacity slider, a press-and-hold "show original" control, a top-3 confidence list, Roboflow-style live thresholds, and a **"Needs field check" state below ~0.6 confidence**. That state maps to Sitemark's "Field Check" action.

States. These are unknown for every commercial product. Tremor ships empty, loading and error primitives, so use them. Example empty state: "No inspections yet: upload drone images".

05 Feature-by-Feature GitHub Mapping

My Feature	Best Existing Project	GitHub	Live Demo	What to Learn
Irradiance ingestion (NASA POWER)	pvlib `iotools.get_nasa_power` (BSD-3)	pvlib-python	None	Signature `get_nasa_power(latitude, longitude, start, end, parameters=['dni','dhi','ghi','temp_air','wind_speed'], community='re', ...)`. **Use it instead of GPL-licensed, stale pynasapower**. The API allows at most 15 parameters per request, and hourly data runs from 2001 (NASA POWER docs)
Clear-sky baseline	pvlib `clearsky.ineichen`, `detect_clearsky`	pvlib-python	None	Use `lookup_linke_turbidity` together with Ineichen, and evaluate deficits only on detected clear periods
Expected AC from irradiance	pvlib `pvsystem.pvwatts_dc`, `inverter.pvwatts`	pvlib-python	None	The chain is irradiance → POA → cell temperature → DC → AC. The gallery example `noplot_oedi_9068.py` models a real OEDI plant
Telemetry ground truth	pvdaq_access (BSD-3)	NREL/pvdaq_access (★11, last push 2025-04-09)	None	`downloadData(system_id, path)` reads the **unsigned** S3 bucket `oedi-data-lake/pvdaq/...` (OEDI 4568)
Data QC + clear-sky labels + PR KPI	pvanalytics (MIT)	pvlib/pvanalytics (★145)	None	`features.clearsky.reno`, `quality.outliers.hampel`, `quality.gaps.completeness_score`, `metrics.performance_ratio_nrel`, `features.clipping.*`, `features.snow.categorize`
Soiling rate from telemetry	RdTools (MIT) `soiling_srr`, `soiling_cods`	NREL/rdtools	None	The stochastic rate-and-recovery method (Deceglie et al. 2018) gives soiling rate, soiling ratio and cleaning intervals, which are **the exact inputs for cleaning ROI**. Also has `degradation_year_on_year` and `AvailabilityAnalysis`
Power-only fallback (no irradiance)	Solar Data Tools (BSD-2)	slacgismo/solar-data-tools	None	`DataHandler.run_loss_factor_analysis`, `detect_clear_days`, `capacity_clustering` (Meyers, arXiv 2209.09457)
Physics-vs-SCADA anomaly app	SolarPulseAI (MIT)	repo	solar-pulse-ai-one.vercel.app (unverified)	Threshold API and soiling-since-rain feature
Inverter peer comparison	pvanalytics outliers + SolarAnalysis notebook	Ritvik-km/SolarAnalysis (no license)	None	Compare each inverter's `DC_POWER/IRRADIATION` to the plant median. This avoids needing an exact model. Treat the notebook as inspiration only
Electrical fault labels	GPVS-Faults (CC BY 4.0)	Mendeley Data	n/a	16 experiments, F0–F7, ~100 kHz V/I. Gives labelled electrical faults that RGB cannot provide
IV-curve diagnosis	pvlib `ivtools.sdm.fit_desoto_sandia`	pvlib-python	None	Rising Rs or falling Rsh across IV sweeps indicates degradation or shunting

My Feature	Best Existing Project	GitHub	Live Demo	What to Learn
6-class RGB classifier	Apollo9 (taxonomy); VisionExpo (TF); HybridSolarNet (benchmark)	Apollo9, VisionExpo	None found	Benchmark target is EfficientNet-B0 at 90.84% and HybridSolarNet at 92.37% / 16.3 MB (arXiv 2601.02928)
Dataset	Kaggle `pythonafroz/solar-panel-images` + mirrors	Linked in EfeCicekdagi README; mirror `kiyoshi732/faulty-solar-panel-images` (ShamSoon)	n/a	**License listed as "Unknown" on Kaggle**, so flag this in the pitch. About 875 raw images
Autoencoder anomaly score	rithvik123 clean-panel anomaly model	repo (no license)	None	Train only on clean panels and use reconstruction error as a heatmap and novelty signal
OOD rejection	SolarScan AI	Princeg1204	streamlit.app URL (unverified)	Reject non-panel images before classifying
LIME superpixels	Apollo9 `explainability.py` (`lime.lime_image` + `skimage.segmentation.slic`); VisionExpo endpoint	explainability.py, VisionExpo	None	SLIC segmentation settings, and the explanation method as an API parameter
Grad-CAM	SolarScan AI; HybridSolarNet Fig. 6	Princeg1204, arXiv	—	Grad-CAM is the **default XAI in PV papers**. For Keras VGG16 it is a short `tf.GradientTape` routine on `block5_conv3` *(assessment)*
Panel localisation in drone images	YOLO11n / YOLOv8-seg + Roboflow datasets	sam3u7858/yolov11n-solar-panel-inspection, Roboflow seg	—	Localise each panel, then classify the crops. A whole-image classifier fails on multi-panel drone frames
Thermal hotspots (future)	InfraredSolarModules; ThermalDetector	RaptorMaps/InfraredSolarModules (MIT, 20,000 images, 12 classes); P-Darabi	—	Electrical faults (diodes, hot-spots) are visible in thermal imagery, not RGB
EL cracks (future)	pv-vision (BSD-3)	hackingmaterials/pv-vision (★60)	—	pip package, weights and pipeline. Datasets: ELPV, PVEL-AD
Cleaning ROI decision	veris-hackathon; Sole_Sight "Loss-to-Cost Ratio"	veris, Sole_Sight	sole-sight.vercel.app (unverified)	Decision enum plus loss, gain, cost and ROI ratio. Rule: clean if LCR > 3:1, otherwise check the rain forecast

My Feature	Best Existing Project	GitHub	Live Demo	What to Learn
Schedule simulation	pvlib `soiling.kimber(..., manual_wash_dates=...)`	pvlib-python	—	Simulate loss under candidate cleaning schedules. Marked experimental
Loss in local currency	solar-om-performance-api spec	bethelhash (spec only)	—	A reference design: IEC 61724-1 weather-corrected PR → 7 root causes → currency loss
Work orders / CMMS	Atlas CMMS (AGPL-3.0 / commercial)	grashjs/cmms	atlas-cmms.com (not loaded)	Work orders, triggers, assets, cost tracking, mobile app. **Run it as a separate service**
Lightweight work orders	solarvision-inspection; Axalon `work_orders.py`	VishnuBhaiya, Axalon	Axalon vercel (unverified)	Completion endpoint, fault-workflow states, completion guards (solar-fleet-ems)
PDF report	Apollo9 `build_pdf_report`; solarvision `pdf-export.js`	Apollo9	—	One-page diagnosis report for a site owner
Weather alternative	openmeteo-requests (MIT)	open-meteo/python-requests	—	Rain forecast for "wait vs clean". The server is AGPL, but calling its API through the MIT client is fine
Dashboard components	Tremor	tremorlabs/tremor	tremor.so (not loaded)	KPI card, BadgeDelta, band charts, states

06 Other Web Projects / Startups / Hackathon Projects

Hackathon, maker and community projects. Three community projects are directly relevant. The **SolarSense** element14 design challenge predicts cleaning time by comparing two test panels against one kept clean, which gives a soiling ratio that cancels out weather. It uses UV, AQI and humidity sensors plus an Edge Impulse classifier that separates dust from clouds, at about **67% accuracy after 30 days**, and targets small installations "where scheduled cleaning isn't economical" (element14). The **veris-hackathon** soiling agent (Section 2.8) and **Sole_Sight's** Loss-to-Cost-Ratio dispatch mock-up (repo) are the closest hackathon analogues to SolarDoctor's ROI loop. Two more were seen only as search titles: an Omdena UAE chapter project on dust detection for cleaning plans (Omdena) and a Contra post titled "AI System for Solar Panel Fault Detection with Live Demo" (Contra). Devpost, Product Hunt and Reddit could not be searched, so Devpost coverage is a known blind spot.

Student-repo saturation. "SolarGuard ... Defect Detection on Solar Panels using Deep Learning" appears in **at least 8 near-identical repos**, which suggests a bootcamp assignment (GitHub search). Examples: Madhugula09/Solar-Panel-Defect-Detection (ResNet-18, ~88–90% claimed) and Mostafakhaled0 (~83%). MainakRepositor/SolarY claims "100%" accuracy with no evaluation details, which almost certainly reflects leakage from augmented duplicates *(assessment)*. Smaller related projects: Anjumkammath/Solarnova-AI (clean vs dusty on a Raspberry Pi, triggering cleaning through GPIO) and rohanmurala/solarsight-ai (EL + ResNet18 + Grad-CAM, with good MODEL_CARD/DATASET_CARD practice).

Hugging Face. No working, verified HF Space for this 6-class taxonomy was found. DSatishchandra/Solar_Panels_Faults_Detection shows a build error. Aumkeshchy2003/Solar_Panel_detection is asleep, was last updated more than a year ago, and detects panels rather than defects. The model Bazaar/cv_solar_panel_defect_detection (ViT) states no classes or accuracy. The HF listing API was blocked, so other Spaces may exist.

Datasets and platforms. Roboflow Universe hosts many RGB defect and segmentation sets, for example segment-solar-cracked-bird_drop and solar-panel-faulty-detection. Their abundance lowers the barrier for anyone to clone SolarDoctor's classifier.

Academic prototypes.

Work	What it shows	Code?
HybridSolarNet (arXiv 2601.02928)	Same 6 classes. EfficientNet-B0 + CBAM at 92.37%, F1 0.9226, 54.9 FPS, 16.3 MB. VGG19 at 87.79% / 532.6 MB. Grad-CAM	No code link
UAV defects dataset (arXiv 2509.05348)	5-class RGB (no snow), 1,633 images. Swin 0.87 mAP vs YOLOv3 0.77. The conclusion and table disagree on numbers	Dataset on GitHub (unverified)
Meyers, SLAC (arXiv 2209.09457)	Soiling from power-only data by signal decomposition. MAE ~0.043. **Weak in cloudy/rainy periods**	Solar Data Tools
Waterloo thesis, Bora (event page)	Cleaning-event detection and an optimal cleaning schedule for a **south-west-India** plant	No code

Work	What it shows	Code?
Bakdi et al., GPVS-Faults (Mendeley); follow-up arXiv 2112.14842	Labelled electrical faults; explainable signature-based fault ID	Dataset (CC BY 4.0)
pv-vision papers (repo)	EL defect identification and crack segmentation	Yes, BSD-3
PV-S3 (repo, arXiv 2404.13693)	Semi-supervised EL segmentation, EAAI 2025	Yes, no license
evelynturri/pv-ai-detector	GPT-4o vs ResNet for PV failures (EUPVSEC 2024)	README not opened

The SolPowNet, KDU and Hohenheim RGB dust-imaging projects were seen as search titles only (SolPowNet, Hohenheim). **No verified paper combines image CV, a physics baseline and XAI in one PV O&M system**, though this search was light.

07 Competitive Landscape

Enterprise inspection and analytics. At utility scale, SolarDoctor's integrated story is already sold commercially. **Raptor Maps** builds digital twins and accepts "input agnostic" inspections from drones, planes, satellites or sensors. It ingests inverter, combiner, pyranometer, SCADA and weather data, quantifies financial and production losses, tracks recurring issues, and normalises to STC for warranty claims, all on a subscription with unlimited inspections (Solar Builder). A secondary Indian source credits it with "90+ fault types" (EAI). **Zeitview** acquired **Heliolytics** in September 2023, which doubled its airplane-inspection reach (Renewable Energy World). Earlier, as DroneBase, it bought India's **AirProbe** (pv magazine India). **Reliance** agreed to buy **79.4% of SenseHawk for about USD 32M** in 2022 (Powerline). So India's largest conglomerate already owns a solar O&M workflow platform.

Indian cleaning robots: the most direct threat to the ROI loop. Pune-based **Taypro** runs waterless robots on **more than 5 GW** and says cleaning is "demand-based... triggered when performance starts to decline". The trigger uses dust, generation trends, humidity, wind and forecasts. Taypro claims 6–15% generation gains and payback in 12–18 months (Mercom India). Another interview gives payback as 2–3 years, so its own figures conflict (now.solar). Its March 2026 roadmap turns robots into "mobile health-monitoring tools" with thermal imaging. Skilancer, Nocca, Solabot, Aegeus and Ecoppia also compete (Renewable Watch). **Fracsun CLEO** models soiling from PM2.5/PM10, wind and precipitation, trained on stations in 27 countries, but it targets developers and has no cleaning-ROI tool (Solar Power World).

Overlap matrix *(assessment built on the verified findings above; "no evidence" means not found, not confirmed absent)*

Player	Physics-baseline deficit	RGB defect CV	Per-image XAI	kWh/₹ loss	Work orders	Fits Indian rooftop/SMB
Raptor Maps	SCADA ingest; method not public	Mainly thermal; input agnostic	No evidence	Yes ($)	Issue tracking	No (enterprise)
Zeitview / Heliolytics	Analytics	Aerial (plane + drone)	No evidence	Power loss % and kW	Status updates	Some rooftop
SenseHawk (Reliance)	Unknown	Drone analytics	No evidence	Unknown	Workflows	Enterprise
Sitemark	Unknown	Thermal + RGB layers	ΔT evidence, not saliency	Est. yield loss + remedial cost	Yes (Field Link)	Unknown
Taypro	Performance-triggered cleaning	Thermal on roadmap	No	% gain claims	Autonomous robot dispatch	Utility / C&I
SolarPulseAI (OSS)	**Yes (pvlib)**	No	No	Revenue loss	No	Indian parks
Apollo9 (OSS)	No	**Yes, same 6 classes**	**LIME / SHAP / Grad-CAM**	No	No	—
SolarDoctor	Yes (NASA POWER / PVDAQ)	Yes (6 classes)	LIME	**₹ ROI**	**Yes**	**Target segment**

Economics constrain the pitch. SECI utility tariffs sit around **₹2.55/kWh** (2019) to **₹2.86/kWh** for solar plus storage (2025) (PV Tech, SolarQuarter). Total O&M runs about **₹2.5 lakh/MW/yr**, and labour, water and tankers can reach **40% of O&M costs** at dusty sites (Renewable Watch, Mercom). As an illustration only, assume 1,500 kWh/kWp/yr and 5% recoverable loss. A 1 MW plant then recovers ~75,000 kWh, about ₹1.9–2.2 lakh/yr, roughly its whole O&M budget. A 10 kW rooftop at an *unverified* ₹7–10/kWh recovers about ₹5,000–7,500/yr. A 2013 UC San Diego study of 186 California sites found only **7.4% loss after 145 dry days**, and about $20 gained from washing a 5 kW system mid-summer. Cleaning paid off mainly for bird droppings, low tilts and sites near dust sources (UC San Diego). That supports positioning SolarDoctor as the tool that says **when cleaning does *not* pay**.

08 What Existing Projects Do Better Than My Idea

Commercial platforms see electrical faults that SolarDoctor cannot. Raptor Maps, Zeitview, Sitemark and Axalon are thermal-first for good reason. Hot-spots, bypass-diode faults and open strings have no reliable RGB signature, and RaptorMaps' public benchmark alone has 11 IR anomaly classes over 20,000 images (InfraredSolarModules). They also map every anomaly to a physical module in a digital twin, so a work order says *which* panel. SolarDoctor's inverter telemetry and single-photo classifier cannot do that.

Their workflows are mature where SolarDoctor's is still a sketch. Raptor has bulk actions and a change log. Zeitview's inspection-to-inspection comparison labels each finding New, Fixed, Persistent, Improved or Degraded (Zeitview). Sitemark has costed remedial actions with field sync (Sitemark). Together these close the loop that SolarDoctor only describes: proving that a fix worked.

The open-source physics libraries are more rigorous than a homemade threshold. RdTools' `soiling_srr` returns soiling rates with **confidence intervals**, and it detects cleaning events and handles precipitation (rdtools). A flat ">5–10% below clear-sky" rule does none of that. Solar Data Tools estimates soiling **without any irradiance data**, removing NASA POWER's grid-resolution error entirely (solar-data-tools). pvanalytics ships the QC (stale values, data shifts, clipping, snow) that a deficit detector needs to avoid false alarms (pvanalytics).

Peer repos beat SolarDoctor on engineering hygiene. SolarScan AI publishes an ablation ladder and **rejects out-of-distribution images** (Princeg1204). HybridSolarNet beats 91.46% at **one-thirtieth the size of a VGG model** (16.3 MB vs 532.6 MB for VGG19) (arXiv 2601.02928). VisionExpo already serves LIME *and* SHAP from a quantised TensorFlow model. solarvision-inspection localises panels before classifying, verifies weight hashes and runs CI. solar-fleet-ems reads eight real inverter clouds, including brands common on Indian rooftops. SolarPulseAI already models soiling from time since rain and inverter clipping for Bhadla and Pavagada.

Taypro already sells the "performance drop triggers cleaning" loop at 5 GW scale (Mercom). SolarDoctor cannot claim that loop as novel for utility plants.

09 What My Idea Can Do Better

First, the weaknesses judges will find. Address them before claiming any advantage.

Weakness	Why it matters	Fix
The 6-class RGB classifier is commoditised	Same taxonomy in dozens of repos; ≥8 SolarGuard clones; EfficientNet/ResNet at 83–90%	Do not lead the pitch with it. Present it as one sensor in a fused system
91.46% is a single number on ~875 source images	Small test sets give wide error bars. Augmented duplicates leaking across splits inflate scores (compare SolarY's "100%"). HybridSolarNet already reports 92.37%	Split *before* augmenting and deduplicate. Report per-class F1, a confusion matrix and a 95% CI. Benchmark against EfficientNet-B0 on the same split *(assessment)*
100×100 grayscale resizing	Colour is the key signal for dust (brown tint), bird-drop (white spots) and snow, and grayscale discards it. VGG16's ImageNet weights expect 224×224 RGB, so grayscale input wastes the pretrained filters, and 100 px erases hairline cracks *(assessment)*	Re-train at 224×224 RGB with VGG16 `preprocess_input`, or move to EfficientNet-B0
RGB-only "Electrical damage"	RGB captures only visible burn marks. Hot-spots, diodes and open circuits need thermal imaging or V/I telemetry (InfraredSolarModules, GPVS-Faults)	Rename the class "Visible burn/discolouration". Route electrical suspicion to the telemetry and IV path (pvlib `fit_desoto_sandia`) and to a "Field check (thermal)" work order
Inverter or string granularity	An inverter-level deficit cannot say which of hundreds of modules is soiled or cracked. The link between "deficit" and "this photo" is therefore assumed, not measured	Store a panel → string → inverter mapping. Make a deficit generate an *inspection task for that string*. Use peer comparison across inverters to localise
Clear-sky baseline vs real weather	Comparing output to *clear-sky* irradiance flags every cloudy hour as a "deficit". NASA POWER is a coarse satellite/reanalysis grid, not a site sensor	Evaluate only on periods that `detect_clearsky` or `reno` marks as clear. Use all-sky NASA POWER GHI to model expected output. Require the deficit to persist for N days
Low utility tariffs	₹2.5–2.9/kWh makes per-site ROI small	Target rooftop and C&I owners, and show "do not clean yet" savings as well as cleaning gains
Dataset license unknown	Kaggle lists the dataset as "Unknown License"	Disclose it. For production, collect your own images

Where SolarDoctor can genuinely be better. These claims are defensible, because each fills a gap the research found rather than resting on "nobody does this".

Fusing telemetry and images to tell clouds from dust. The Meyers soiling method fails in cloudy periods (arXiv 2209.09457), and SolarSense needed a physical clean reference panel to solve the same problem (element14). A persistent clear-sky-period deficit *confirmed by a photo* classified as Dusty is stronger

evidence than either signal alone. No inspected repo does this, and none uses NASA POWER or PVDAQ inside an app.

Measured explanations. No solar vendor documents per-image saliency. Only Apollo9 and VisionExpo ship LIME, and **no opened repo evaluates explanation quality**. Pairing LIME with Grad-CAM and reporting a deletion/insertion faithfulness check would put SolarDoctor ahead on XAI rigour. Showing measured evidence (deficit %, kWh) next to the heatmap, Sitemark's "explanation by measurement", adds further credibility.

Rupee decisions for an underserved segment. Enterprise tools do not target Indian rooftop and C&I owners, and Indian third-party O&M is "highly fragmented" (Renewable Watch). A ₹-denominated clean / wait / field-check decision, using RdTools soiling rates rather than illustrative numbers, would beat solarvision-inspection's self-described "illustrative" ROI.

A working public demo. No verified working HF Space exists for this taxonomy, so a live Gradio or Streamlit Space is itself a differentiator.

An autoencoder + VGG16 combination. No inspected repo uses it, but being new is not the same as being better. Justify it with a measured gain, for example the autoencoder catching unseen anomaly types as OOD, or drop it.

10 Top 5 Code References

#	Reference	License	Exactly what to read	Why
1	pvlib/pvlib-python (★1712, pushed 2026-11-03, v0.16.1)	BSD-3	`iotools.get_nasa_power`, `clearsky.ineichen` / `detect_clearsky`, `pvsystem.pvwatts_dc`, `inverter.pvwatts`, `soiling.kimber`, gallery `docs/examples/system-models/noplot_oedi_9068.py`	The whole expected-yield engine in one maintained, permissive library
2	NREL/rdtools (v3.2.1, 2026-07-08)	MIT	`soiling.soiling_srr`, `normalize_with_expected_power`, `filtering.pvlib_clearsky_filter`, notebook `degradation_and_soiling_example.ipynb`	Turns the deficit into a soiling rate with uncertainty, which is what ROI needs
3	mcomuniello/Apollo9	**None (read only)**	`explainability.py`, `inference.py`, `main.py`	Same 6 classes with LIME, SHAP and Grad-CAM behind FastAPI. Reimplement it; do not copy
4	VisionExpo/solar-panel-fault-detection	Apache-2.0	FastAPI explanation endpoint, model quantisation, Render/Docker config	A TensorFlow stack match with a reusable license
5	Parakh20/AxalonSystems	MIT	`platform/api/routers/work_orders.py`, `platform/core/fault_workflow.py`, Alembic setup	A reusable, production-style fault-to-work-order workflow

Runners-up: pvanalytics `docs/examples/pvfleets-qa-pipeline/` for QC, and rithvik123 `dashboard/api_client.py` for the Streamlit-to-FastAPI pattern (study only).

11 Top 5 UI/UX References

#	Reference	Take this specific pattern
1	Sitemark anomaly analysis + remedial actions	Anomaly drawer section order, remedial-action vocabulary, "Estimated Remedial Cost", Swipe Compare
2	Raptor Maps Digital Twin	Dashboard/Map/List tri-view, live count bubbles, bulk action, change log
3	Zeitview severity & comparison	3-tier severity, % loss → 5-grade condition, compare-status colour legend
4	Home Assistant Energy cards	Forecast-over-actual chart (draw it as an expected band with a shaded deficit), global date picker with compare-to-previous
5	Tremor	KPI cards with delta badges, plus empty, loading and error states (the React phase)

Honourable mention: Roboflow's live confidence-threshold slider for the diagnosis screen. All five were observed through documentation only, not in a live UI.

12 Top 3 Repositories to Use as a Foundation

Legal ground rules. Repos with **no license** grant no reuse rights, even when public. Apollo9, rithvik123, solarvision-inspection, solar-fleet-ems, veris-hackathon, SolarScan AI, Ritvik-km/SolarAnalysis and the Kaggle-dataset notebooks are **not legally reusable**; read them for ideas and write your own code. **AGPL-3.0** projects (Atlas CMMS, solectrus, OpenEMS, the Open-Meteo server) require you to release your source if you modify them and offer them over a network. So run them unmodified as separate services behind an API, and get legal review before commercial use. GPL projects (RentadroneCL, pynasapower) carry copyleft obligations, so avoid embedding them.

#	Foundation	License	Use it for	What must change
1	**pvlib-python** + **rdtools** + **pvanalytics**	BSD-3 / MIT / MIT	The entire pillar (a): NASA POWER ingestion, clear-sky/PVWatts expected output, QC, soiling rate, PR	Nothing in the libraries. Write a thin `baseline_service.py` around them. Note that pvlib soiling models are marked experimental, and that Kimber/Hsu defaults are not tuned for Indian dust (no verified source). Keep the copyright notices
2	**VisionExpo/solar-panel-fault-detection**	Apache-2.0	Pillars (b)+(c) skeleton: TF model behind FastAPI, LIME/SHAP endpoint, Gradio UI, Docker/Render	Swap in your VGG16+AE (retrained at 224×224 RGB), or adopt its EfficientNet, and compare honestly. Add Grad-CAM, an OOD gate and a low-confidence "Field check" state. Retain LICENSE/NOTICE and mark modified files, as Apache-2.0 requires. Its training dataset license is also unclear
3	**SolarPulseAI** (or AxalonSystems for work orders)	MIT	The app shell for telemetry + alerts (FastAPI, TimescaleDB, threshold API, Indian plant seeds). Axalon supplies the work-order router and fault workflow	Replace the LightGBM forecast with the pvlib/NASA POWER baseline (or keep it as a comparison). Add image-diagnosis and work-order tables. Its frontend is Next.js, so either keep it for the React phase or call only its backend from Streamlit. Keep the MIT notice. Verify the code actually runs, since the repos look AI-assisted and were not executed here

For dispatch beyond the hackathon, **Atlas CMMS** (AGPL-3.0 or commercial, dual-licensed since 2026-01-08) can run in its own container via `docker-compose up -d`, with SolarDoctor posting work orders to its REST API. Its API schema was not checked.

13 Recommended Development Strategy Based on This Research

The strategy is to **lead with the fused loop, not the classifier**. Each phase below produces a demoable artifact, so the team can stop at any phase and still have something to show.

Phase 0: make the model numbers defensible (hours 0–3). Re-split the Kaggle data by source image *before* augmentation and check for near-duplicates across splits. Re-train VGG16 at **224×224 RGB** with `tf.keras.applications.vgg16.preprocess_input`. Train EfficientNet-B0 on the identical split as the honest baseline. Report per-class precision, recall and F1, a confusion matrix and a bootstrap 95% CI, published as an ablation table like SolarScan AI's. Keep the autoencoder only if it measurably helps, for example as an OOD/novelty gate trained on Clean images. Rename "Electrical damage" to "Visible electrical burn". Save models as `.keras` and quantise them for hosting, following VisionExpo.

Phase 1: physics deficit engine (hours 3–8). Write `baseline_service.py`. It should pull a PVDAQ system with `pvdaq_access.downloadData` and irradiance with `pvlib.iotools.get_nasa_power`, transpose to POA, then run `pvwatts_dc` → `inverter.pvwatts`. Run QC with pvanalytics (gaps, outliers, clipping). Label clear periods with `detect_clearsky` or `pvanalytics.features.clearsky.reno`. Flag a deficit only when `(expected − measured)/expected > 5–10%` on clear periods and it persists for several days. Feed the daily normalised performance index into `rdtools.soiling.soiling_srr` to get the soiling rate and cleaning events. Add per-inverter peer comparison (Hampel or z-score) to localise the fault to one inverter. Display a Zeitview-style condition grade from % loss.

Phase 2: fusion and ₹ ROI (hours 8–12). A flagged inverter or string creates an **inspection task**. The uploaded photo's class plus the deficit then decides the action: Dusty + deficit → Clean, Physical crack → Replace or Warranty, low confidence or a suspected electrical fault → Field check (thermal). Compute recoverable kWh from the soiling rate × expected energy, then ₹ = kWh × a user-entered tariff, set against the cleaning cost. Return veris-style fields: projected loss, gain, cost, ROI ratio and the decision (`clean_now` / `wait` / `field_check`). Optionally simulate schedules with `pvlib.soiling.kimber(manual_wash_dates=...)`. Present the `T* ≈ sqrt(2·C/(r·E·tariff))` interval as an approximation, not a cited result. Use an Open-Meteo rain forecast for "wait, rain is coming".

Phase 3: work orders (hours 12–15). Use a SQLite/SQLModel `work_orders` table with statuses **To Do → In Progress → Resolved / Won't Fix**, Sitemark's action vocabulary, estimated cost, assignee, a before/after photo pair and a change-log table. Add a completion guard, as in solar-fleet-ems: a work order cannot resolve without an after-photo. Close the loop by showing the deficit chart recovering after the cleaning date. Export a one-page PDF report. Defer Atlas CMMS integration.

Phase 4: Streamlit UI (hours 15–20). Build pages for Overview (KPI cards: deficit %, ₹ at risk, open work orders), Site Performance (actual line over a clear-sky band with the deficit shaded), Diagnose (upload → OOD gate → top-3 confidence bars → LIME and Grad-CAM tabs with an opacity slider and original/explanation swipe → ROI card → "Create work order") and Work Orders (board). Keep heatmap colours separate from severity colours. Streamlit must call FastAPI through an `api_client.py` (the rithvik123 pattern) so React can replace it later.

Phase 5: ship and pitch (hours 20–24). Run FastAPI and Streamlit as two containers with docker-compose, and deploy a Gradio or Streamlit **Hugging Face Space**, since no working one exists for this taxonomy. For the pitch, cite pvlib, RdTools and Solar Data Tools as prior art. Position SolarDoctor as "the rupee decision layer for

rooftop/C&I owners that tells clouds from dust". Be upfront that RGB cannot see hot-spots and that the dataset license is unknown. Judges reward candour over "unique" claims in a commoditised space.

After the hackathon. Migrate the UI to React + Tremor with the Raptor/Sitemark drawer IA. Add YOLO11-seg panel localisation on Roboflow data for drone frames, a thermal branch (InfraredSolarModules / ThermalDetector), and inverter-cloud adapters for Growatt and Solis following solar-fleet-ems's structure, rewritten rather than copied. Add an explanation-faithfulness evaluation (deletion/insertion curves) suitable for a paper.

Conclusion

The research changes where SolarDoctor's value lies. The CV module, which most hackathon teams would showcase, is the most replicated part of the idea. The physics baseline already exists as mature, permissive NREL and pvlib tooling, so it is a reuse decision rather than an invention. What remains unclaimed in open source is the joint inference: a clear-sky-period deficit, confirmed or rejected by a photo, priced in rupees, and closed out by a verified work order. Doing that well also fixes the known weakness of production-only soiling methods, their inability to separate clouds from dirt.

The most credible version of SolarDoctor is therefore a smaller and more honest one. It runs RGB at full resolution with error bars and routes electrical suspicion to telemetry or thermal checks rather than RGB guesses. It localises faults at string level first, and frames ROI as "when *not* to clean" as much as when to clean. At SECI-level tariffs the economics are thin. The pitch should therefore aim at C&I and rooftop owners, where tariffs are higher and enterprise vendors are absent, and where Taypro-style robots are uneconomical.

Sources

GitHub: apps and full-stack projects

- github.com/mcomuniello/Apollo9 (inference.py, explainability.py, app.py)
- github.com/anugrahpatil-creator/SolarPulseAI
- github.com/VishnuBhaiya/solarvision-inspection
- github.com/rithvik123/solar-panel-defect-detection
- github.com/Parakh20/AxalonSystems
- github.com/VisionExpo/solar-panel-fault-detection
- github.com/toannnnq1424/solar-fleet-ems
- github.com/jkumarr10/veris-hackathon
- github.com/Princeg1204/solar-panel-defect-classifier
- github.com/dunglt69/solar-anomaly-detection
- github.com/Mostafakhaled0/Solar-Panel-Defect-Classifier
- github.com/Madhugula09/Solar-Panel-Defect-Detection
- github.com/bethelhash/solar-om-performance-api
- github.com/lke-2/guardian-p
- github.com/xenofex7/solar-tracker
- github.com/purvask2006-collab/Sole_Sight
- github.com/P-Darabi/SolarPanelsDefectDetector
- github.com/4keles/Solar-Panel-AI-Analysis
- github.com/solectrus/solectrus
- github.com/BorisBrock/Sunalyzer
- github.com/OpenEMS/openems
- github.com/EfeCicekdagi/Solar-Panel-Fault-Detection-with-YOLOv11-nano
- github.com/rahaffayez/ShamSoon_Solar_Panel_Fault_Detection
- github.com/MainakRepositor/SolarY
- github.com/suhasmaddali/Solar-Panels-Dust-Detection
- github.com/Sanatnagpal/Solar-Panel-Fault-Detection
- github.com/rohanmurala/solarsight-ai
- github.com/Anjumkammath/Solarnova-AI
- github.com/Ritvik-km/SolarAnalysis
- github.com/sam3u7858/yolov11n-solar-panel-inspection
- github.com/search?q=Bird-drop+Electrical-damage&type=repositories

GitHub: libraries, datasets, CMMS

- github.com/pvlib/pvlib-python
- github.com/pvlib/pvanalytics
- github.com/NREL/rdtools
- github.com/slacgismo/solar-data-tools
- github.com/NREL/pvdaq_access
- github.com/NREL/pysam
- github.com/alekfal/pynasapower
- github.com/open-meteo/python-requests
- github.com/grashjs/cmms
- github.com/hackingmaterials/pv-vision
- github.com/zae-bayern/elpv-dataset
- github.com/binyisu/PVEL-AD
- github.com/RaptorMaps/InfraredSolarModules
- github.com/abj247/PV-S3
- github.com/tremorlabs/tremor

Data and APIs

- power.larc.nasa.gov/docs/services/api/temporal/hourly
- data.openei.org/submissions/4568
- data.mendeley.com/datasets/n76t439f65/1
- universe.roboflow.com/thesis-nns8j/solar-panels-instance-segmentation
- universe.roboflow.com/6rainstorm-yqytq/solar-panel-faulty-detection

Papers

- arxiv.org/html/2601.02928v1 (HybridSolarNet)
- arxiv.org/html/2509.05348v1 (UAV defects dataset)
- arxiv.org/pdf/2209.09457 (Meyers, soiling from power data)
- arxiv.org/pdf/2112.14842 (GPVS-Faults explainable ML)
- arxiv.org/abs/2404.13693 (PV-S3)
- uwaterloo.ca/computer-science/events/masters-thesis-presentation-data-systems-data-driven-estimation-soiling-loss-optimal-cleaning-schedule-for-utility-scale-pv-plant

UI/UX documentation

- pages.raptormaps.com/raptor-maps-knowledge-hub/using-the-digital-twin-for-your-sites
- raptormaps.com/blog-posts/what-s-new-at-raptor-maps
- user-guide.insights.zeitview.com
- user-guide.insights.zeitview.com/solar-comparison-view-9
- support.sitemark.com/en/articles/4825953-analyzing-results-solar-inspections
- support.sitemark.com/en/articles/6029063-managing-remedial-actions-for-thermal-anomalies
- support.sitemark.com/en/articles/13010251-filtering-for-faster-solar-anomaly-decisions
- home-assistant.io/lovelace/energy
- docs.redwoodjs.com/docs/how-to/build-dashboards-fast-with-tremor
- docs.roboflow.com/models/rapid/rapid/deploy-a-rapid-model

www4.enphase.com/en-au/support/what-does-it-mean-have-public-view-my-system-0
- themoonlight.io/en/review/interactivity-x-explainability-toward-understanding-how-interactivity-can-improve-computer-vision-explanations

Industry and market

solarbuildermag.com/news/raptor-maps-adds-unlimited-inspections-and-more-handy-reporting-tools-to-raptor-solar
renewableenergyworld.com/solar/utility-scale/zeitview-acquires-heliolytics-expanding-aerial-solar-inspections
pv-magazine-india.com/?p=4840
powerline.net.in/2022/09/07/ril-to-acquire-79-4-per-cent-stake-in-us-based-sensehawk-for-usd32-million
eai.in/blog/2025/07/how-ai-is-revolutionizing-om-in-utility-scale-solar-power-plants.html
solarpowerworldonline.com/2024/07/fracsun-unveils-new-ai-powered-solar-soiling-loss-modeling-tool
solarpowerworldonline.com/2024/10/power-factors-taps-ai-new-asset-management-offerings
mercomindia.com/robotic-cleaning-cuts-om-costs-by-40-boosts-solar-generation-by-up-to-15-interview
timestech.in/how-taypros-ai-robots-are-redefining-solar-plant-performance
now.solar/2026/03/14/the-future-of-solar-cleaning-robots-is-as-mobile-health-monitoring-tools-for-solar-plants-yogesh-kudale-taypro-saur-energy
- renewablewatch.in/2024/11/08/optimising-operations-current-practices-and-new-opportunities-in-solar-om
- pv-tech.org/secis-1-2gw-ists-solar-auction-draws-lowest-bid-of-2-55-rupees
- solarquarter.com/2025/10/16/seci-concludes-2000-mw-solar-and-1000-mw-4000-mwh-energy-storage-auction-at-%e2%82%b92-86-per-kwh
- today.ucsd.edu/story/cleaning_solar_panels_often_not_worth_the_cost_engineers_at_uc_san_diego_fi
- community.element14.com/challenges-projects/design-challenges/on-the-line/b/projects/posts/solarsense---edge-ai-for-optimal-cleaning-of-solar-panels
- omdena.com/chapter-challenges/machine-learning-approach-to-detect-dust-on-solar-panels-in-uae-a-contribution-toward-optimizing-cleaning-plan
- huggingface.co/spaces/DSatishchandra/Solar_Panels_Faults_Detection
- huggingface.co/spaces/Aumkeshchy2003/Solar_Panel_detection/blob/main/README.md